

# VisionFlow

Real-Time Object Detection, Multi-Object Tracking  
and Video Analytics with YOLOv4-tiny and OpenCV

## Project Report

CSE3010 - Computer Vision

VITyarthi: Build Your Own Project

Submitted by **Tushar Chakraborty**

Registration No: **24BAI10842**

Date: 07 October 2026

Repository: <https://github.com/Tushar5467/VisionFlow>

## 1. Overview

VisionFlow is a command-line video analytics tool developed as the project deliverable for CSE3010 (Computer Vision). It runs a pre-trained YOLOv4-tiny detector through OpenCV's DNN module, follows each detected object from frame to frame with a centroid-distance tracker, and keeps a directional tally of objects that cross a virtual line. On top of the neural detections it adds two classical techniques, Canny edge detection and MOG2 background subtraction with an accumulating motion heatmap.

The scope is deliberately limited to **inference only**. No model is trained or fine-tuned anywhere in the code. This keeps the work appropriate for a single-course project while still exercising ideas from several syllabus modules: filtering and enhancement (Module 1), edge and feature extraction (Module 3) and motion analysis with background modelling (Module 4).

## 2. Problem Definition

Watching recorded footage by hand to count people or vehicles, to see where movement concentrates, or to find busy areas is tedious and easy to get wrong. Two automated shortcuts each fall short. A motion detector based only on background subtraction reacts to anything that changes, so it cannot tell a pedestrian from a shadow or a moving branch. A deep detector run independently on each frame can say what is visible right now, but it has no memory, so it cannot answer questions about identity such as how many different people walked across a given line.

VisionFlow closes this gap by pairing a compact detector with classical tracking and motion analysis. The result is persistent object IDs, direction-aware counts and machine-readable reports, all produced from a terminal with no GUI and no training stage.

## 3. Functional Requirements

The system is organised into four functional modules, each with a defined input and output:

| # | Module                                             | Input                            | Output                                                   |
|---|----------------------------------------------------|----------------------------------|----------------------------------------------------------|
| 1 | Object Detection (YOLODetector)                    | One BGR video frame              | List of Detection objects (box, class, confidence)       |
| 2 | Tracking (CentroidTracker, LineCrossingCounter)    | Detections for the current frame | Object ID to box mapping; up/down crossing counts        |
| 3 | Classical Analytics (EdgeDetector, MotionAnalyzer) | One video frame                  | Canny edge map; foreground mask; decaying motion heatmap |
| 4 | Reporting (ReportGenerator)                        | Tracked detections per frame     | detections.csv (audit log) and summary.json (totals)     |

**Typical run:** the user starts the CLI with a video file or webcam index. Each frame is passed in turn through detection, tracking, analytics and reporting. Annotations are drawn, the annotated frame is written to the output video, and when the stream ends the CSV, JSON and console summaries are produced.

## 4. Non-Functional Requirements

- **Performance:** the detector is wrapped in a `@timeit` decorator, average FPS is stored in `summary.json`, and a `PROCESS_EVERY_N_FRAMES` setting lets slower machines skip frames in exchange for speed.

- **Reliability:** model files are checked before loading and a clear message is shown if they are missing; unreadable video sources end gracefully instead of throwing a stack trace; frames with no detections are handled normally.
- **Usability:** one documented entry point (main.py) with subcommands and --help text, and no GUI dependency, which meets the course requirement that the project run from the command line.
- **Maintainability:** each module has one job (detector, tracker, analytics, report generator, pipeline) and every tunable value lives in config.py rather than being scattered as magic numbers.
- **Logging:** timestamped, levelled, module-tagged log lines go to both the console and output/logs/visionflow.log.
- **Resource use:** inference is CPU-only and uses the 23 MB YOLOv4-tiny network instead of the 245 MB full YOLOv4, with an optional frame-resize step to cap per-frame cost.
- **Security and privacy:** video never leaves the local machine. The only network use is the explicit, one-time download-model command, which fetches public model files from GitHub.

## 5. System Architecture

VisionFlow uses a layered pipeline design. A light CLI layer parses arguments and hands control to a Pipeline Orchestrator. The orchestrator drives four independent components (Detector, Tracker, Analytics, Reporter), all of which share one configuration module and one logging setup.

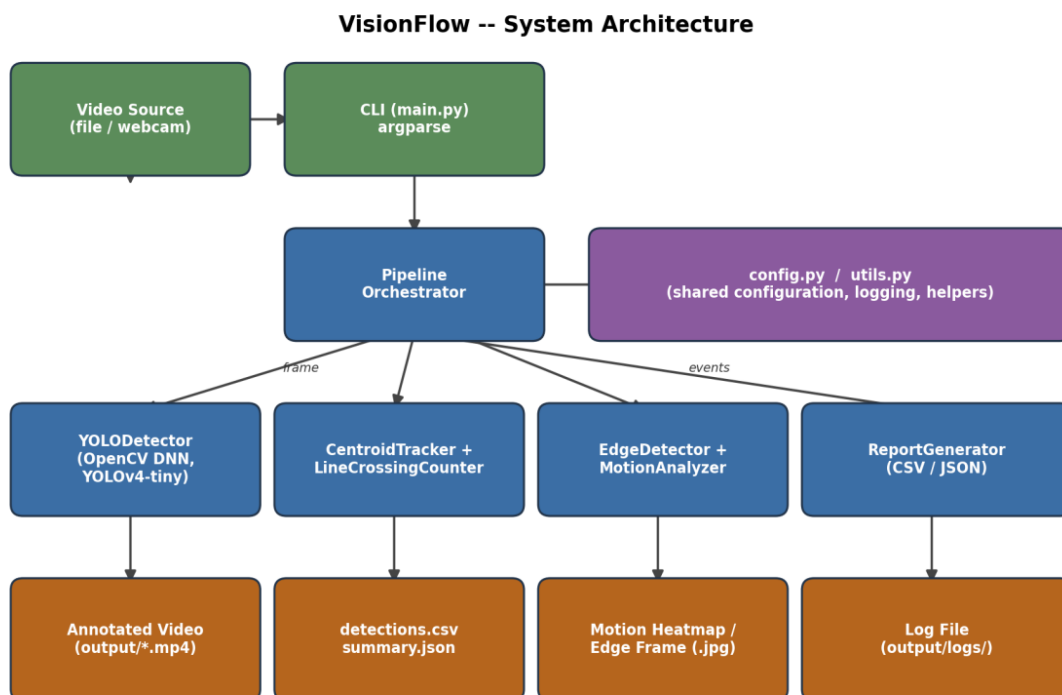


Figure 1: High-level architecture of VisionFlow.

## 6. Design Diagrams

### 6.1 Use Case Diagram

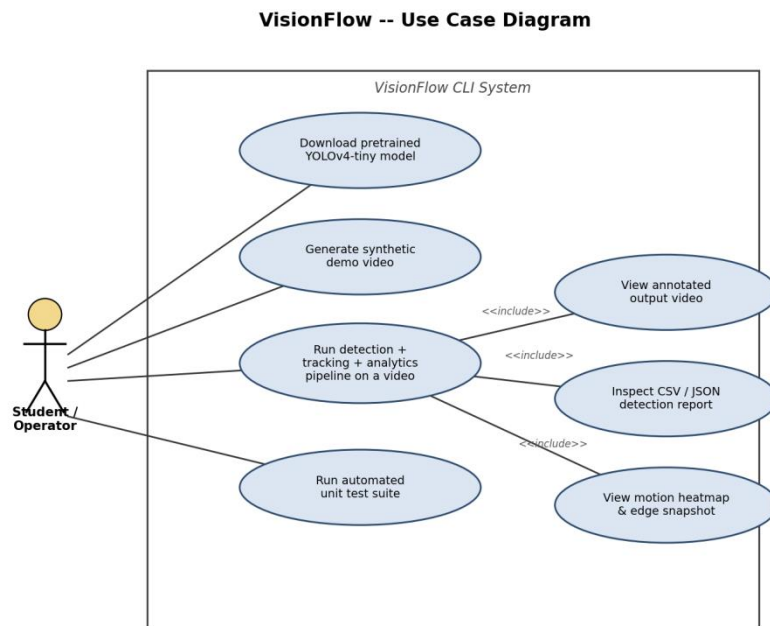


Figure 2: Use cases available to the Student/Operator actor.

### 6.2 Workflow Diagram

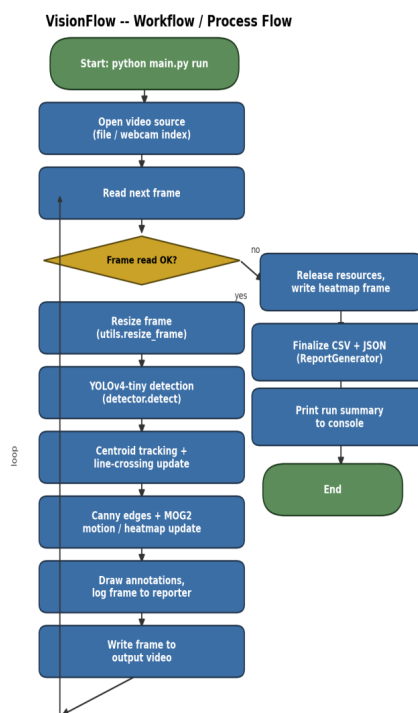


Figure 3: End-to-end run, including the per-frame loop.

## 6.3 Sequence Diagram

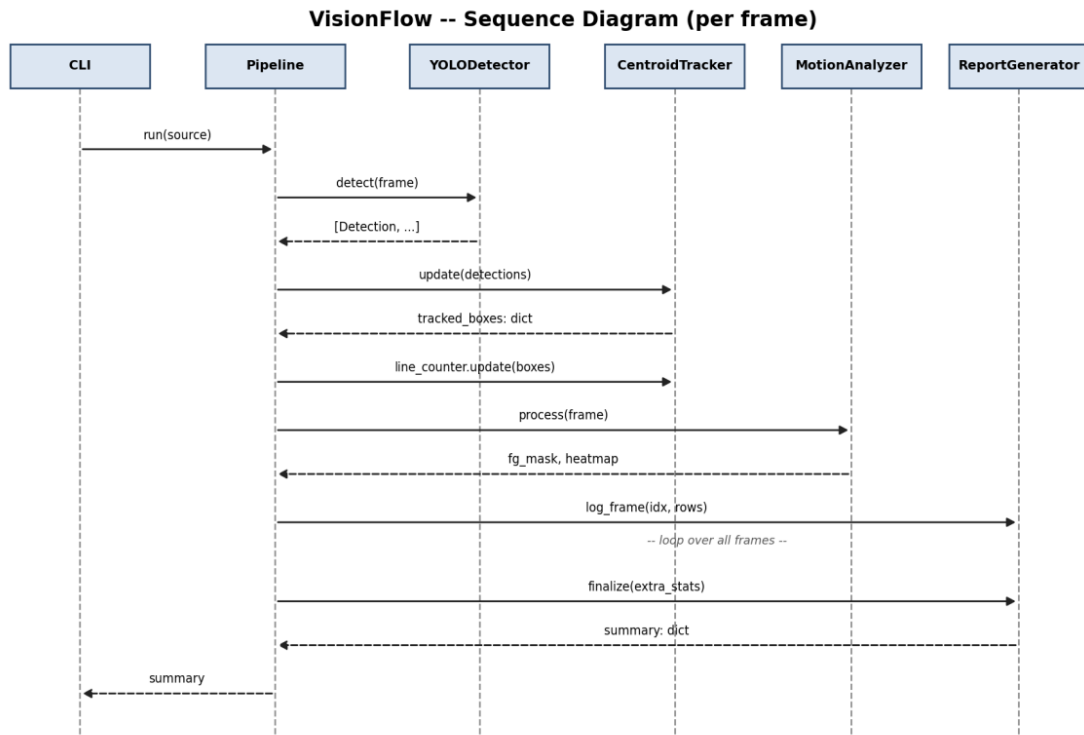


Figure 4: Calls exchanged between the Pipeline and its components for each frame.

## 6.4 Class Diagram

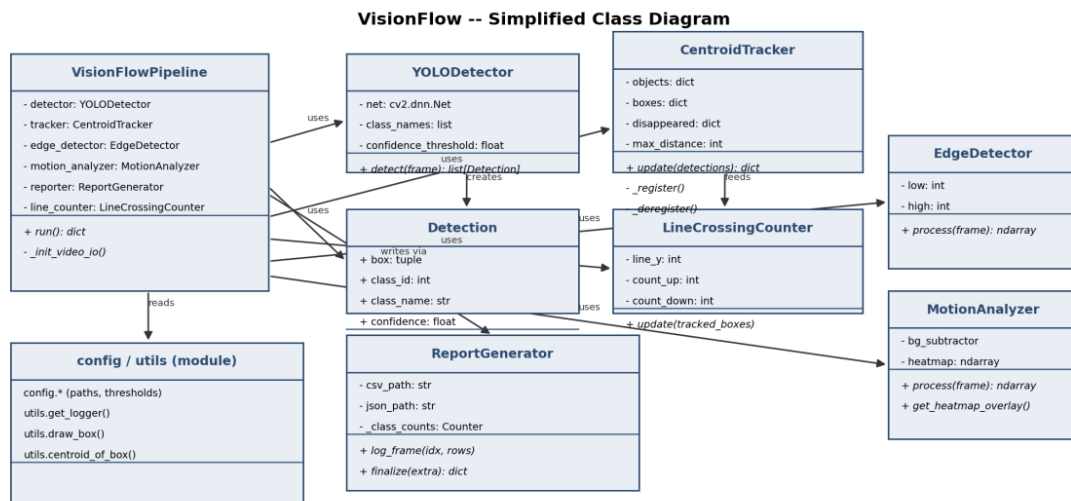


Figure 5: Simplified class diagram of the main modules.

## 7. Design Decisions and Justification

### 7.1 YOLOv4-tiny rather than full YOLOv4

The tiny variant reaches workable frame rates on a CPU, which is what most student machines and the grading environment offer, and it still recognises all 80 COCO categories. Full YOLOv4 would be much slower on a CPU without a matching gain for a project of this size.

### 7.2 A centroid tracker instead of learned re-identification

Matching by centroid distance is easy to follow, needs no extra model, and is adequate for one camera viewing a scene of moderate density. A learned re-ID network would add complexity out of proportion to what the project needs to show.

### 7.3 Classical analytics next to a deep detector

The syllabus names filtering (Module 1), edge extraction (Module 3) and background subtraction with motion analysis (Module 4). Adding Canny and MOG2 applies those topics directly, so the project does not rest on the neural network alone.

### 7.4 OpenCV DNN instead of a separate deep-learning framework

`cv2.dnn.readNetFromDarknet` loads the Darknet weights using only OpenCV and NumPy. Nothing like PyTorch or TensorFlow has to be installed, which makes setup easy for evaluators and honours the "OpenCV and YOLO only" constraint.

### 7.5 Model and dataset choice

The detector is YOLOv4-tiny, pre-trained by the Darknet authors on MS COCO (80 classes, roughly 330K images). Because the task is inference only, no dataset was collected, and no domain-specific fine-tuning was done. The general COCO training already covers everyday subjects such as people, vehicles and animals.

### 7.6 How correctness was checked

- **Unit tests** cover each module in isolation, including a fully mocked detector path that needs no model download.
- **Reference-image check:** real inference was run on AlexeyAB/darknet's `dog.jpg`, and the expected classes (dog, bicycle, truck) were found with reasonable confidence.
- **End-to-end run:** a 150-frame pedestrian video was processed to confirm that detection, tracking, analytics and reporting work together and write consistent, inspectable outputs.

## 8. Implementation Details

### 8.1 Detection (`src/detector.py`)

`YOLODetector` wraps `cv2.dnn.readNetFromDarknet`. For every frame it builds a 416x416 input blob (scaled to [0,1], BGR converted to RGB), runs a forward pass through the output layers, discards results under a configurable confidence threshold, and removes overlapping duplicates with Non-Maximum Suppression (`cv2.dnn.NMSBoxes`).

## 8.2 Tracking (src/tracker.py)

CentroidTracker stores a dictionary of object IDs and their centroids. On each update it uses NumPy to compute Euclidean distances between every existing centroid and every new detection centroid, then greedily pairs the closest ones that fall within a maximum distance. Unpaired detections become new objects. Unpaired existing objects have a 'disappeared' counter incremented and are removed once it passes a configurable limit. LineCrossingCounter separately remembers each object's previous vertical position and counts a directional crossing the first time the centroid passes a configured horizontal line.

## 8.3 Classical analytics (src/analytics.py)

EdgeDetector applies a Gaussian blur and then Canny edge detection. MotionAnalyzer wraps `cv2.createBackgroundSubtractorMOG2`, keeps only true foreground pixels (the shadow label is excluded), and builds a floating-point heatmap that fades over time and is displayed as a JET colormap overlay.

## 8.4 Reporting (src/report\_generator.py)

ReportGenerator writes detection events to the CSV as they happen, so memory use stays flat even on long videos. Per-class totals are kept with `collections.Counter`, and a JSON summary (FPS, elapsed time, line-crossing counts) is written when the run ends.

## 8.5 Command line (main.py)

The argparse CLI has three subcommands: `download-model` fetches the weights, `cfg` and class names; `generate-demo` makes a synthetic test video; and `run` executes the full pipeline on a file or webcam index, with optional `--show` and `--no-save` flags.

# 9. Results

First, the detector was checked on a known reference photograph to confirm that the model loads and performs real inference rather than merely running without errors.

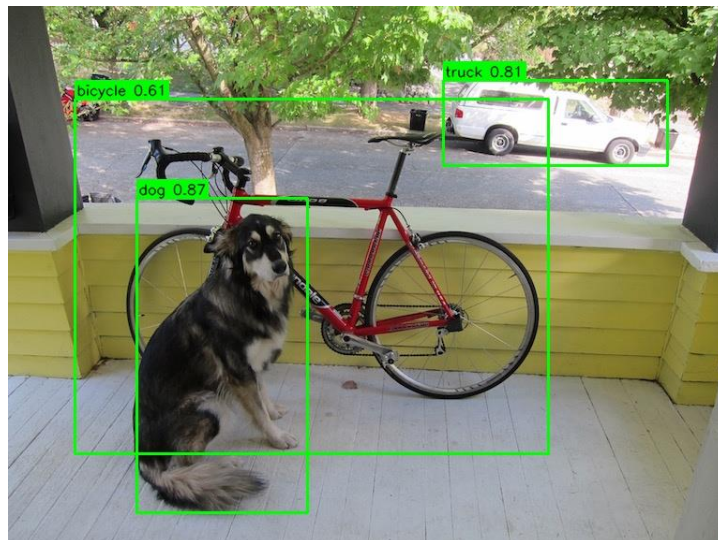


Figure 6: YOLOv4-tiny detecting dog (0.87), truck (0.81) and bicycle (0.61) in the reference image.

The complete pipeline was then run on a real 150-frame pedestrian clip from the Intel IoT DevKit sample-videos collection.



Figure 7: A tracked person with a persistent ID, confidence score and the counting line.

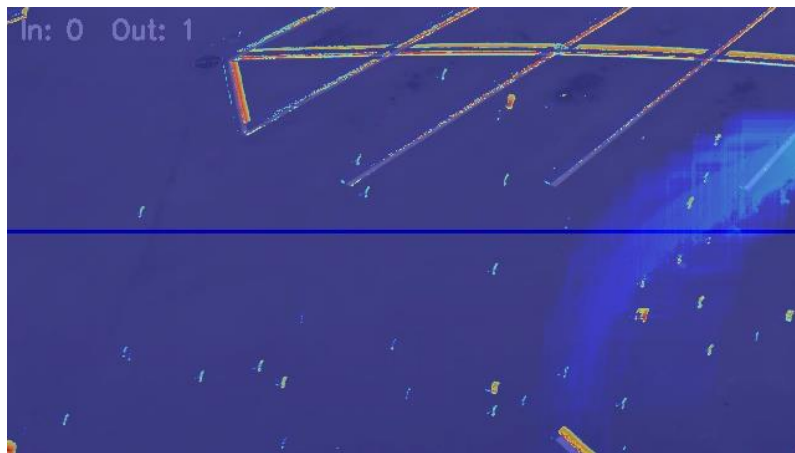


Figure 8: Motion heatmap (JET colormap) accumulated over the same run.

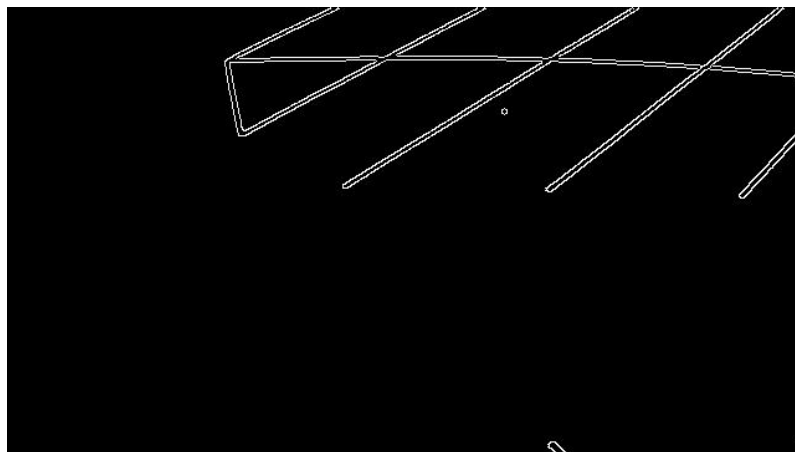


Figure 9: Canny edge map from an early frame of the same run.

**Run summary (output/summary.json):**

| Metric                      | Value              |
|-----------------------------|--------------------|
| Frames processed            | 150                |
| Average FPS (CPU inference) | about 8.9          |
| Detections logged           | 62 (class: person) |
| Line crossings (out)        | 1                  |
| Processing time             | about 17 seconds   |

## 10. Testing

The repository ships 18 pytest unit tests, grouped by module:

- **test\_tracker.py (6):** registering new objects, keeping IDs across frames, deregistering after a timeout, assigning new IDs to distant detections, and directional line-crossing logic.
- **test\_analytics.py (6):** Canny output shape and dtype, edges on high-contrast input, no edges on a blank frame, lazy heatmap initialisation, foreground-mask value range, and heatmap response to a moving object.
- **test\_report\_generator.py (3):** CSV header, row and summary consistency over several frames, and inclusion of extra run-level stats.
- **test\_detector.py (3):** a clear `FileNotFoundError` when model files are absent, parsing and NMS against a stubbed `cv2.dnn` network (so the suite runs offline without the 23 MB download), and confidence-threshold filtering.

In addition, the system was integration-tested twice on real data: once with a static photograph whose expected classes were known, and once as a complete CLI run on a pedestrian video, after which all five outputs (video, CSV, JSON, heatmap, edge sample) were inspected by hand. Running `pytest tests/ -v` reports **18 passed**.

## 11. Problems Encountered

- **Finding the weights:** the first GitHub release tag tried (`darknet_yolo_v3_optimal`) did not contain the YOLOv4-tiny weights. They were located under the `yolov4` release tag.
- **Heatmap crash at end of video:** when the final read returned `None`, the heatmap step failed. The fix was to keep a separate reference to the last successfully read frame.
- **MOG2 tuning:** the variance threshold had to be raised enough to ignore compression noise and static scene lines, yet stay sensitive to real movement.
- **Linking tracker output to metadata:** to recover class name and confidence for each tracked box without a heavier per-object structure, a dictionary keyed on the exact box tuple returned by the tracker is used for each frame.

## 12. Key Learnings

- Running a pre-trained Darknet YOLO model purely through OpenCV's DNN module, with no deep-learning framework.

- Seeing how plain centroid-distance matching already gives useful object persistence without re-identification.
- Benefits of single-responsibility modules with shared configuration, which made isolated testing with a mocked detector easy.
- The practical sensitivity-versus-noise trade-off in MOG2 that the motion-analysis module describes in theory.

## 13. Future Work

- A lightweight re-identification embedding to keep identity through longer occlusions.
- Several user-defined counting lines or zones in place of one horizontal line.
- CUDA backend support in OpenCV DNN for higher frame rates.
- SQLite storage so results from many runs can be queried together.
- A read-only web dashboard for viewing summaries and heatmaps.

## 14. References

- Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934.
- AlexeyAB/darknet repository (YOLOv4-tiny weights, cfg, COCO names). <https://github.com/AlexeyAB/darknet>
- OpenCV documentation, cv2.dnn module. <https://docs.opencv.org/>
- Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV.
- Intel IoT DevKit sample-videos (person-bicycle-car-detection.mp4). <https://github.com/intel-iot-devkit/sample-videos>
- CSE3010 Computer Vision course material, Modules 1, 3 and 4.